# Pertemuan 2
# Fungsi, Subprogram, dan Rekursi

## Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

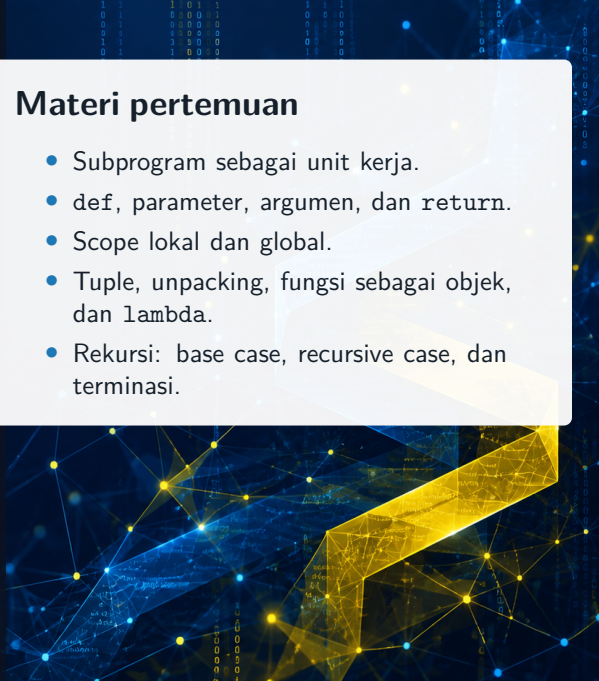
Triyana Muliawati, S.Si., M.Si.
Rifky Fauzi

Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

### Materi pertemuan

- Subprogram sebagai unit kerja.
- `def`, parameter, argumen, dan `return`.
- Scope lokal dan global.
- Tuple, unpacking, fungsi sebagai objek, dan `lambda`.
- Rekursi: base case, recursive case, dan terminasi.



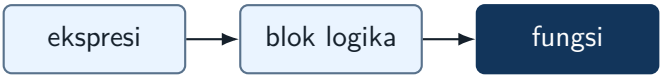
# Dari ekspresi ke fungsi

## Yang sudah dikuasai

- Membuat ekspresi dari rumus matematika.
- Menggunakan variabel untuk menyimpan nilai.
- Mengatur alur dengan `if`, `for`, dan `while`.

## Langkah berikutnya

- Memberi nama pada proses yang sering dipakai.
- Menentukan input dan output secara jelas.
- Menulis program sebagai susunan unit kecil.



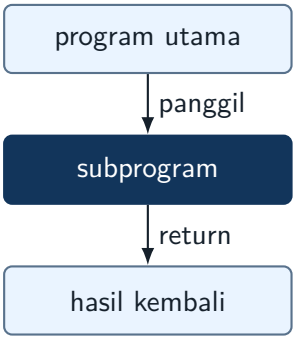
# Apa itu subprogram?

## Makna umum

Subprogram adalah bagian program yang diberi nama untuk menjalankan satu tugas tertentu. Subprogram dapat dipanggil dari bagian lain program.

## Dalam Python

Bentuk utama subprogram adalah fungsi yang ditulis dengan `def`. Fungsi dapat menerima input melalui parameter dan mengirim hasil melalui `return`.




Subprogram membuat program lebih mudah dibaca, diuji, dan digunakan ulang.

# Fungsi sebagai aturan transformasi

$$y = f(x)$$

input $\rightarrow$ proses $\rightarrow$ output




## Dalam matematika

Fungsi menghubungkan masukan dengan keluaran menurut aturan tertentu.

Dalam program, fungsi menyimpan aturan itu dalam bentuk langkah-langkah yang dapat dijalankan komputer.

## Dalam Python

`def` memberi nama pada aturan komputasi. Parameter menjadi masukan. `return` menjadi keluaran.

# Anatomi fungsi Python

```
def nama_fungsi(parameter_1, parameter_2):
    langkah_1
    langkah_2
    return hasil
```

## Bagian definisi

- `def`: mendefinisikan fungsi.
- Nama fungsi: tugas yang dikerjakan.
- Parameter: nama input di dalam fungsi.

## Saat dipanggil

- Argumen: nilai yang dikirim ke fungsi.
- `return`: nilai yang dikirim kembali.
- Setelah `return`, fungsi selesai.

# Contoh: fungsi dari rumus matematika

$$p(x) = 2 - 3x + x^2$$

$$p(4) = 2 - 12 + 16 = 6$$

```
def polinom_sederhana(x):
    return 2 - 3*x + x**2

hasil = polinom_sederhana(4)
print(hasil)
```

**Pertanyaan membaca kode**

Nilai mana yang menjadi parameter?
Nilai mana yang menjadi argumen?

# Parameter dan argumen

```
def luas_persegi_panjang(panjang, lebar):
    return panjang * lebar

A = luas_persegi_panjang(5, 3)
B = luas_persegi_panjang(lebar=3, panjang=5)
```

**Positional argument**

Urutan argumen menentukan parameter yang menerima nilai.

**Keyword argument**

Nama parameter disebutkan langsung. Urutan tidak lagi menjadi sumber utama kesalahan.

Untuk fungsi matematis, nama parameter yang jelas mengurangi salah tafsir.

# Fungsi tanpa argumen

## Tanpa argumen, tanpa `return`

```
def salam():
    print("Selamat belajar")

salam()
```

Cocok untuk menampilkan pesan atau instruksi.

## Tanpa argumen, dengan `return`

```
def konstanta_pi():
    return 3.14159

r = konstanta_pi()
```

Cocok untuk nilai tetap atau konfigurasi.

# Fungsi dengan argumen

## Dengan argumen, tanpa `return`

```
def tampil_kuadrat(x):
    print(x**2)

tampil_kuadrat(5)
```

Hasil tampil di layar, tetapi tidak bisa langsung dipakai untuk hitungan lanjutan.

## Dengan argumen, dengan `return`

```
def kuadrat(x):
    return x**2

y = kuadrat(5) + 2
```

Ini bentuk yang paling sering dipakai dalam komputasi matematika.

# `print` tidak sama dengan `return`

```python
def g(x):
    return x**2 - 1

def h(x):
    print(g(x))

nilai = h(3)
print(nilai)
```

```python
z = h(3) + 2
# TypeError
```

**Ide utama**

`print` hanya menampilkan nilai. Jika tidak ada `return`, fungsi mengembalikan `None`.

# Fungsi yang mengembalikan lebih dari satu nilai

```
def stat_ringkas(a, b, c, d):
    nilai = [a, b, c, d]
    minimum = min(nilai)
    maksimum = max(nilai)
    rata = sum(nilai) / len(nilai)
    rentang = maksimum - minimum
    return minimum, maksimum, rata, rentang

mn, mx, r, rg = stat_ringkas(4, 7, 1, 9)
```

## Yang terjadi

Python sebenarnya mengembalikan satu objek bertipe tuple. Unpacking membagi isi tuple ke beberapa variabel.

# Tuple dan unpacking

```
hasil = stat_ringkas(4, 7, 1, 9)
print(hasil)
print(type(hasil))

minimum, maksimum, rata, rentang = hasil
```

```
# jumlah variabel harus cocok
x, y = hasil
# ValueError
```

**Mengapa penting?**

Satu proses sering menghasilkan beberapa informasi: nilai estimasi, galat, jumlah iterasi, atau status konvergensi.

# Scope: variabel lokal dan global

```
x = 10

def f(a):
    x = a + 2
    return x

print(f(5))
print(x)
```

## Variabel lokal

x di dalam fungsi hanya hidup di dalam fungsi tersebut.

## Variabel global

x di luar fungsi tetap bernilai 10. Nilai ini tidak otomatis berubah hanya karena ada x lokal.

# Mengapa variabel global perlu hati-hati?

```
total = 0

def tambah(x):
    global total
    total = total + x
    return total
```

## Masalah yang sering muncul

- Hasil fungsi bergantung pada riwayat pemanggilan.
- Test menjadi tidak mandiri.
- Kesalahan sulit dilacak karena nilai dapat berubah dari tempat lain.

Utamakan fungsi yang menerima input secara eksplisit dan mengembalikan output secara eksplisit.

# Fungsi sebagai alat dekomposisi

$$d(p,q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

$$E(p,q) = \frac{1}{1 + d(p,q)^2}$$

```
def jarak(p, q):
    dx = p[0] - q[0]
    dy = p[1] - q[1]
    return (dx**2 + dy**2)**0.5

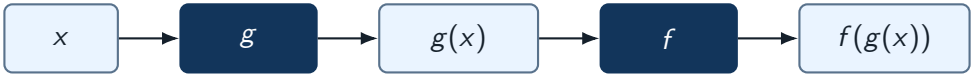
def energi(d):
    return 1 / (1 + d**2)

def energi_dua_titik(p, q):
    return energi(jarak(p, q))
```

# Komposisi fungsi

Jika $g$ menerima $x$, dan $f$ menerima hasil dari $g$, maka

$$(f \circ g)(x) = f(g(x)).$$




## Dalam Python

Ide yang sama muncul saat satu fungsi memanggil fungsi lain. Program menjadi susunan unit kecil, bukan satu blok panjang.

# Fungsi sebagai objek

```
def proses(x, f):
    return f(x)

def kuadrat(x):
    return x**2

def tambah1(x):
    return x + 1

print(proses(5, kuadrat))
print(proses(5, tambah1))
```

## Makna penting

Nama `kuadrat` tanpa tanda kurung adalah objek fungsi. Objek itu dapat dikirim sebagai argumen ke fungsi lain.

# List fungsi dan pipeline

```
def kuadrat(x):
    return x**2

def tambah1(x):
    return x + 1

def pipeline(x, ops):
    hasil = x
    for f in ops:
        hasil = f(hasil)
    return hasil

ops = [abs, kuadrat, tambah1]
print(pipeline(-3, ops))
```

## Baca alurnya

Nilai $-3$ diproses oleh `abs`, lalu `kuadrat`, lalu `tambah1`. Pola ini sering muncul pada transformasi data.

# Lambda: fungsi pendek untuk operasi pendek

$$f(x) = 3x^2 - 2x + 1$$

```
f = lambda x: 3*x**2 - 2*x + 1
```

```
xnilai = range(-3, 4)

y = [f(x) for x in xnilai]

z = list(map(lambda x: 3*x**2 - 2*x + 1,
             xnilai))
```

## Gunakan secara wajar

Lambda cocok untuk fungsi satu baris yang jelas. Jika logika mulai panjang, fungsi bernama lebih baik.

# List comprehension untuk evaluasi fungsi

```
def f(x):
    return 3*x**2 - 2*x + 1

xnilai = [-3, -2, -1, 0, 1, 2, 3]
y = [f(x) for x in xnilai]
```

## Cara membaca

Untuk setiap `x` di `xnilai`, hitung `f(x)`, lalu simpan hasilnya ke list baru.

List dipakai sebagai wadah data sederhana. Struktur data dan array NumPy dibahas lebih formal pada pertemuan berikutnya.

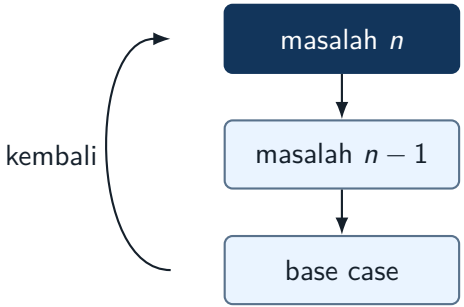
# Rekursi: fungsi yang memanggil dirinya sendiri

## Ide utama

Masalah dipecah menjadi versi yang lebih kecil dari masalah yang sama.

## Dua bagian wajib

- **Base case**: kondisi berhenti.
- **Recursive case**: pemanggilan fungsi pada masalah yang lebih kecil.



# Contoh rekursi: faktorial

$$n! = n(n-1)!, \qquad 0! = 1.$$

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1.$$

```
def faktorial(n):
    if n == 0:
        return 1
    return n * faktorial(n - 1)

print(faktorial(5))
```

## Telusuri

Tuliskan urutan pemanggilan `faktorial(5)`, `faktorial(4)`, ..., sampai kembali menghasilkan nilai akhir.

# Rekursi untuk barisan

Diberikan barisan

$$a_0 = 2, \qquad a_n = 3a_{n-1} + 1, \quad n \geq 1.$$

```
def a(n):
    if n == 0:
        return 2
    return 3*a(n - 1) + 1
```

**Tugas kecil**

Hitung manual $a_1$, $a_2$, dan $a_3$.

**Yang perlu diamati**

Setiap pemanggilan bergerak dari $n$ ke $n-1$, sehingga mendekati base case.

# Rekursi dan terminasi

## Berhenti

```
def turun(n):
    if n == 0:
        return 0
    return turun(n - 1)
```

## Tidak bergerak ke kondisi berhenti

```
def salah(n):
    if n == 0:
        return 0
    return salah(n + 1)
```

Pertanyaan penting: apakah ukuran masalah benar-benar mengecil menuju kondisi berhenti?

# Diagnosa kesalahan fungsi

```
def rata(data):
    total = 0
    for x in data:
        total = total + x
        return total / len(data)

print(rata([2, 4, 6, 8]))
```

## Pertanyaan

- Mengapa hasilnya salah walaupun sintaksnya valid?
- Di mana posisi `return` seharusnya?
- Buat dua test sederhana untuk memeriksa fungsi setelah diperbaiki.

# Checklist membaca fungsi

Saat melihat fungsi baru, tanyakan hal berikut.

1. Apa tugas yang dikerjakan fungsi?
2. Apa input yang dibutuhkan?
3. Apa output yang dikembalikan?
4. Apakah fungsi hanya mencetak, atau benar-benar mengembalikan nilai?
5. Apakah fungsi bergantung pada keadaan di luar dirinya?
6. Jika rekursif: apa base case dan recursive case-nya?

Untuk mahasiswa Matematika, fungsi dapat dipahami sebagai padanan komputasional dari transformasi, norma, jarak, ringkasan, atau operator.

# Latihan akhir 1: menulis fungsi

Tulis fungsi berikut dengan input dan output yang jelas.

1. `polinom(x, koef)` untuk menghitung $a_0 + a_1 x + \cdots + a_n x^n$.
2. `stat_ringkas(a,b,c,d)` yang mengembalikan minimum, maksimum, rata-rata, dan rentang.
3. `jarak(p,q)` untuk jarak Euclid dua titik di bidang.

**Syarat**

Setiap fungsi harus dipanggil minimal dua kali. Tuliskan contoh input, output yang diharapkan, dan hasil yang diperoleh Python.

# Latihan akhir 2: diagnosa kesalahan

Periksa fungsi berikut. Jelaskan kesalahannya, perbaiki, lalu uji.

```
def norm1(v):
    total = 0
    for x in v:
    total = total + abs(x)
    print(total)

hasil = norm1([3, -4, 1, 2]) + 5
```

## Yang harus ditemukan

Indentasi, perbedaan `print` dan `return`, serta apakah nilai fungsi dapat dipakai untuk operasi lanjutan.

# Latihan akhir 3: membuat fungsi rekursif

Diberikan barisan Fibonacci

$$F_0 = 0, \qquad F_1 = 1, \qquad F_n = F_{n-1} + F_{n-2}.$$

1. Tulis fungsi rekursif `fib(n)`.
2. Tentukan base case dan recursive case.
3. Telusuri pemanggilan untuk `fib(5)`.
4. Bandingkan jumlah pemanggilan dengan versi iteratif untuk $n$ kecil.

# Quiz e-learning M2

## Materi quiz

- Membaca definisi fungsi dan memprediksi output.
- Membedakan parameter dan argumen.
- Membedakan `print`, `return`, dan `None`.
- Menelusuri scope lokal dan global.
- Menentukan hasil unpacking.
- Mengenali base case dan recursive case.

## Cara belajar

Prediksi dahulu di kertas atau catatan. Setelah itu jalankan Python untuk memeriksa dan menuliskan koreksi pemahaman.

# Tugas mandiri M2

## Yang dikumpulkan

1. Empat bentuk fungsi sesuai materi.
2. Fungsi `polinom(x, koef)`.
3. Fungsi `stat_ringkas` dengan unpacking.
4. Contoh scope lokal dan global beserta penjelasan.
5. `proses(x, f)` dan `pipeline(x, ops)`.
6. Satu fungsi rekursif untuk barisan sederhana.

## Format

Boleh menggunakan Google Colab, Jupyter, Spyder, VS Code, atau IDE Python lain. Kode harus dapat dijalankan ulang dan diberi komentar singkat.

# Penutup

## Hal yang perlu dikuasai

- Membuat fungsi dengan input dan output jelas.
- Menggunakan `return` secara tepat.
- Menelusuri scope.
- Mengirim fungsi sebagai argumen.
- Menulis rekursi yang memiliki kondisi berhenti.

## Arah pertemuan berikutnya

- Representasi algoritma.
- List, tuple, dan dictionary.
- Pseudocode.
- Array NumPy dan comprehension.

Fungsi yang baik membuat program lebih dekat dengan cara berpikir matematis: jelas inputnya, jelas transformasinya, jelas outputnya.